

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО»**

**Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці**

**Розрахунково-графічна робота
ПРОЄКТУВАННЯ ТА СПЕЦИФІКАЦІЯ
ПРОСТОЇ ІМПЕРАТИВНОЇ МОВИ ПРОГРАМУВАННЯ
ЗАГАЛЬНОГО ПРИЗНАЧЕННЯ
(з дисципліни «Основи розробки трансляторів»)
Мова програмування Krok
Рівень 2 Основний**

Учасники проєкту:

Степаненко Назар Юрійович
Аніщенко Артем Олександрович

Виконав: студент III курсу, групи ТВ-43
спеціальності 121 «Інженерія програмного забезпечення»

Степаненко Назар Юрійович

(Прізвище, ім'я, по батькові)

— — — — — (Підпис)

Перевірів: доцент, к.т.н. Стативка Ю.І.

(Посада, науковий ступінь, прізвище та ініціали)

— — — — — (Підпис)

Київ – 2026

ЗМІСТ

1 Вступ	4
1.1 Назва мови	4
1.2 Рівень складності та мова-прототип	4
1.3 Призначення та ключові особливості	5
1.4 Обробка програми	6
1.5 Нотація	7
1.5.1 Розширена нотація Бекуса — Наура	7
1.5.2 Граматика мови у канонічній формі	7
1.5.3 Синтаксичні діаграми	8
1.5.4 Представлення семантики	9
1.6 Відповідність вимогам завдання	9
2 Лексична структура	11
2.1 Алфавіт	11
2.2 Роздільники рядків та пробільні символи	17
2.3 Коментарі	17
2.4 Лексеми і токени	18
2.5 Ідентифікатори	19
2.6 Ключові слова	20
2.7 Літерали	21
2.7.1 Цілі літерали	22
2.7.2 Дійсні літерали	22
2.7.3 Рядкові літерали	23
2.7.4 Логічні літерали	24
2.8 Оператори та роздільники	25

2.9 Автоматична вставка крапки з комою.....	28
2.10 Таблиця токенів	29

1 Вступ

Цей документ є специфікацією мови програмування **Krok** — формальним і повним описом її лексики, синтаксису, системи типів та семантики. Структуру документа побудовано за переліком елементів специфікації імперативної мови, наведеним у розд. 2.4 навчального посібника [1]: вступ, лексична структура, типи даних і змінні, вирази, інструкції, декларації, структура програми, повна граматики мови. Додатково наведено розділ про властивості розробленої граматики та розділ контрольних прикладів програм.

1.1 Назва мови

Мова має назву **Krok**; назва вимовляється як «крок» і записується в текстах латиницею. Назва є транслітерацією українського слова «крок» і пов'язана з мовою-прототипом: англійське *go* означає «іди», а крок — елементарна одиниця руху. Так само кожна інструкція програми є одним кроком алгоритму, а сама мова є «кроком» від повноцінної мови Go до простої навчальної мови, для якої в межах дисципліни буде розроблено транслятор.

Файли з вихідним кодом мовою Krok мають розширення `.krok`.

1.2 Рівень складності та мова-прототип

Мова Krok відповідає **Рівню 2 (Основний)**: це процедурно-модульна статично типізована мова з користувацькими типами. Об'єктно-орієнтовані концепції не підтримуються.

Мовою-прототипом обрано мову **Go** (https://go.dev/doc/effective_go). З мови Go запозичено синтаксис оголошень (`var`, `const`, `type`, `func`), структуру програми (`package main`, `import "fmt"`, функція `main`), інструкцію присвоєння (`=`) та оголошення з ініціалізацією (`:=`), інкремент і декремент (`++`, `--`), інструкції повторення `for` та розгалуження `if-else`, введення/виведення через пакет `fmt`, однорядкові коментарі `//`, правило автоматичної вставки крапки з комою, масиви, структури та вказівники як механізм передачі параметрів за посиланням.

Семантику елементів мови адаптовано до потреб навчального транслятора. Основні відмінності від мови-прототипу наведено в табл. 1.1.

Таблиця 1.1 – Основні відмінності мови Krok від мови-прототипу Go

Аспект	Go	Krok
Приведення <code>int</code> → <code>float</code>	лише явне	неявне розширення та явне приведення
Піднесення до степеня	немає оператора	оператор <code>**</code> , правоасоціативний
Переповнення <code>int</code>	циклічне (<code>wrap-around</code>)	помилка часу виконання
Типи <code>int</code> , <code>float</code> , <code>bool</code> , <code>string</code>	попередньо оголошені ідентифікатори	ключові слова
Вказівники	довільні значення, <code>nil</code>	лише типи параметрів функцій
Складені літерали	для масивів, зрізів, структур, мап	лише для масивів із явним типом <code>[N]T</code>
Інструкція <code>for</code>	чотири форми, <code>break</code> , <code>continue</code>	дві форми: з передумовою та трикомпонентна
<code>if</code> з ініціалізацією, <code>switch</code>	є	немає
Зрізи, мапи, методи, інтерфейси, горутини	є	немає
Пакети	довільні	лише <code>main</code> та вбудований <code>fmt</code>

Проект виконується спільно (виконавці — Степаненко Н. Ю. та Аніщенко А.). Відповідно до вимог до спільних проєктів кожен виконавець обирає власний варіант інструкцій повторення та розгалуження. У цій специфікації описано варіант: **інструкція `for` з передумовою та трикомпонентна інструкція `for`** (розд. 5.4) і **інструкція `if` з необов'язковими гілками `else if` та `else`** (розд. 5.3).

1.3 Призначення та ключові особливості

Krok — проста імперативна процедурна мова програмування загального призначення, статично і суворо типізована, призначена для запису обчислювальних алгоритмів, що взаємодіють із середовищем через стандартні потоки введення та виведення: числові розрахунки, обробка масивів, рекурсивні алгоритми, прості моделі даних на основі структур.

Ключові особливості мови:

- статична типізація: тип кожної змінної та кожного виразу відомий на етапі трансляції;
- сувора типізація: єдиним неявним перетворенням є розширення `int` → `float`, решта перетворень — явні;
- обов'язкове оголошення імен і нульові значення для неініціалізованих змінних;
- семантика значень для масивів і структур; передача за посиланням лише через явні параметри-вказівники;
- відсутність динамічного розподілу пам'яті та збирача сміття: уся пам'ять — статична область і стек викликів;
- перевірка меж масивів та контроль переповнення чисел під час виконання;
- граматики мови належить до класу LL(1), тому транслятор будується методом рекурсивного спуску з одним токеном попереднього перегляду.

1.4 Обробка програми

Програма мовою Krok подається на вхід транслятора (компілятора або інтерпретатора) у вигляді одного текстового файлу в кодуванні UTF-8. Трансляція складається з таких фаз:

1. Лексичний аналіз — окремий прохід, що перетворює потік символів (`character stream`) на потік токенів (`token stream`), вилучає коментарі та пробільні символи і вставляє неявні крапки з комою (розд. 2.9). Уся інформація, необхідна для побудови лексичного аналізатора, міститься в розд. 2.
2. Синтаксичний аналіз — перевірка відповідності потоку токенів граматиці мови та побудова дерева розбору (`parse tree`).
3. Семантичний аналіз — перевірка контекстних умов: оголошеність імен, області видимості, сумісність типів тощо.
4. Генерування коду або безпосереднє виконання.

Результат трансляції виконується системою часу виконання (run-time system), яка отримує вхідні дані зі стандартного потоку введення, записує результати у стандартний потік виведення, а повідомлення про помилки — у стандартний потік помилок.

1.5 Нотація

1.5.1 Розширена нотація Бекуса — Наура

Для опису лексики та синтаксису мови Krok використовується розширена нотація Бекуса — Наура (РБНФ) у варіанті Н. Вірта, описаному в розд. 2.2.2 посібника [1]. Метасимволи нотації наведено в табл. 1.2.

Таблиця 1.2 – Прийнята нотація РБНФ

Метасимвол	Значення
$A = \alpha$	нетермінал A визначається як α
$\alpha \mid \beta$	альтернатива: α або β
$[\alpha]$	0 або 1 екземпляр α
$\{ \alpha \}$	0 або більше екземплярів α
$(\alpha \mid \beta)$	групування
Abc	нетермінал (починається з великої літери)
abc	термінал — ключове слово (починається з малої літери)
'+', 'Scan'	термінал, записаний у лапках буквально
'\t', '\n', '\r'	термінали: символи табуляції, нового рядка та повернення каретки
? текст ?	спеціальна послідовність: неформально описана множина символів
(* текст *)	коментар до граматики

Правило починається з імені нетерміналу на початку рядка; рядки, що починаються з пробілів, продовжують попереднє правило. Обраний варіант РБНФ не використовує ані символу завершення правила, ані знака конкатенації: послідовність символів позначається їх записом поруч.

1.5.2 Граматика мови у канонічній формі

Формально граматика мови Krok є контекстно вільною граматикою $G = (N, T, P, S)$, де N — множина нетерміналів, T — множина терміналів, P — множина продукцій виду $\alpha \rightarrow \beta$, S — аксіома (початковий символ). Алфавітом

граматики є $V = N \cup T$, а алфавітом самої мови — множина терміналів T . Оскільки лексичний аналіз виконується окремим проходом, граматику поділено на дві частини:

- **синтаксична граMATика** з аксіомою `Program` описує, як із токенів утворюються програми; її терміналами є токени мови (розд. 2.10);
- **лексична граMATика** з аксіомою `SourceText` описує, як із символів алфавіту утворюються лексеми; її терміналами є символи алфавіту (розд. 2.1).

Нетермінали `Ident`, `IntLit`, `FloatLit` та `StringLit` є в синтаксичній граматиці неподільними токенами; нетермінали `BoolLit`, `RelOp`, `AssignOp` та `IncDecOp` групують токени і використовуються в обох частинах. Кожне правило має рівно один нетермінал у лівій частині, отже граMATика мови є контекстно вільною (тип 2 за класифікацією Хомського).

Лексика мови, як і лексика мов програмування загалом (розд. 2.1 у [1]), належить до вузького класу **регулярних** мов: жодне лексичне правило не є самовкладеним, тому потік символів розпізнається детермінованим скінченним автоматом, а не магазинним. Обидві частини записано однією нотацією РБНФ в одному файлі, тобто формально вони утворюють єдину контекстно вільну граматику мови. Кількісні характеристики граматики та результати її перевірки наведено в розд. 8.

1.5.3 Синтаксичні діаграми

Для графічного представлення граматики використовуються синтаксичні діаграми Вірта. Відповідно до розд. 2.2.4 посібника [1] для кожного правила граматики побудовано окрему діаграму у формі орієнтованого графа: термінали зображено в овалах (прямокутниках із заокругленими кутами), нетермінали — у прямокутниках (курсивом), спеціальні послідовності — курсивом без рамки. Вхід і вихід діаграми орієнтовано зліва направо; ланцюжок належить мові, якщо він описує шлях від входу до виходу.

Діаграми згенеровано автоматично з файлу граматики `grammar/krok.ebnf` програмою `tools/gen_diagrams.py` (бібліотека `railroad-diagrams`), тому кожне правило має рівно одну діаграму з ідентичними назвами символів.

Ту саму граматику експортовано в нотації всіх трьох генераторів діаграм, згаданих у розд. 2.2.4 посібника [1] (програма `tools/export_visualizer.py`), тому будь-яке правило мови можна відобразити кожним із цих інструментів:

- `grammar/krok-visualizer.ebnf` — EBNF Visualizer [6];
- `grammar/krok-iso14977.ebnf` — EBNF 2 RailRoad (нотація ISO/IEC 14977);
- `grammar/krok-w3c.ebnf` — Railroad Diagram Generator (нотація W3C).

1.5.4 Представлення семантики

Семантику мови представлено **неформально** — природномовним текстом, який супроводжує опис синтаксичних конструкцій, з таблицями типів, операторів і помилок та з прикладами програм (розд. 9). Такий спосіб опису відповідає розд. 2.3.2 посібника [1] і практиці специфікацій мов Python, Go та Pascal; формальних моделей (операційної, денотаційної чи аксіоматичної семантики) ця специфікація не використовує.

Опис семантики кожної конструкції вказує, за яких умов її виконання **завершується нормально**, а за яких виникає помилка. Перелік помилок наведено в розд. 7.3 (виявляються транслятором) та 7.4 (виявляються під час виконання).

1.6 Відповідність вимогам завдання

Таблиця 1.3 – Відповідність специфікації вимогам завдання

Вимога	Конструкція мови Krok	Розділ
Оператор циклу	<code>for умова { }, for init; умова; post { }</code>	5.4
Оператор розгалуження	<code>if умова { } else if умова { } else { }</code>	5.3
Цілі та дійсні числа, + - * /	типи <code>int</code> , <code>float</code> ; оператори + - * / %	3.2, 4.4.1
Піднесення до степеня (правоасоціативне)	оператор <code>**</code>	4.4
Логічний тип	<code>bool</code> , <code>true</code> , <code>false</code> , <code>&&</code> <code> </code> <code>!</code>	3.2, 4.4.3

Вимога	Конструкція мови Krok	Розділ
Рядковий тип	string, рядкові літерали	3.2, 4.4.4
Іменовані константи	const	6.2
Порівняння	== != < <= > >=	4.4.2
Дужки у виразах	(Expression)	4.2
Однорядкові коментарі	// ...	2.3
Приведення int ↔ float	неявне розширення, float(x), int(x)	3.6.1, 3.6.2
Введення/виведення	fmt.Scan, fmt.Print, fmt.Println	5.5
Функції, рекурсія	func, return	6.4
Передача за значенням і за посиланням	параметри T та *T, &x, *p	3.4.3
Локальні та глобальні змінні	блоки, затінення	6.5
Користувачькі скалярні типи	type Celsius float	3.3
Масиви (багатовимірні, статична та динамічна ініціалізація)	[N]T, [N]T{...}	3.4.1
Структури (факультативно)	type P struct { ... }	3.4.2
Модульна структура (факультативно)	import "fmt"	7.1

2 Лексична структура

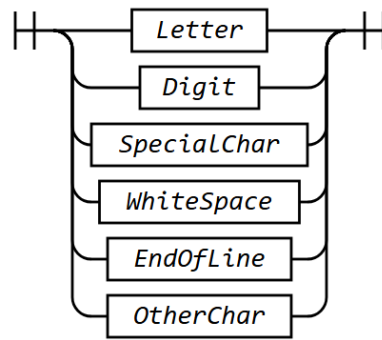
Лексичний аналіз виконується окремим проходом і не залежить від синтаксичних та семантичних аспектів мови. Лексичний аналізатор читає текст програми зліва направо і на кожному кроці виділяє **найдовшу** послідовність символів, що утворює лексему (правило найдовшого збігу). Наприклад, ****** розпізнається як один оператор піднесення до степеня, а не як два оператори множення; **<=** — як один оператор, а **a---b** — як **a**, **--**, **-**, **b**.

2.1 Алфавіт

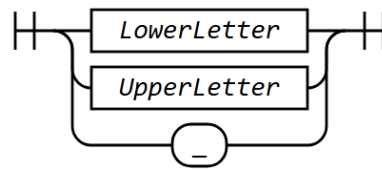
Текст програми мовою Krok записується в кодуванні UTF-8. Поза рядковими літералами та коментарями програма може містити лише латинські літери, цифри, спеціальні знаки, пробільні символи та символи кінця рядка. Усередині рядкових літералів і коментарів допускаються будь-які символи Unicode (зокрема кирилиця), крім символів кінця рядка. Мова чутлива до регістру літер: **total**, **Total** і **TOTAL** — різні ідентифікатори.

Alphabet	= Letter Digit SpecialChar WhiteSpace EndOfLine OtherChar
Letter	= LowerLetter UpperLetter '_'
LowerLetter	= 'a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i' 'j' 'k' 'l' 'm' 'n' 'o' 'p' 'q' 'r' 's' 't' 'u' 'v' 'w' 'x' 'y' 'z'
UpperLetter	= 'A' 'B' 'C' 'D' 'E' 'F' 'G' 'H' 'I' 'J' 'K' 'L' 'M' 'N' 'O' 'P' 'Q' 'R' 'S' 'T' 'U' 'V' 'W' 'X' 'Y' 'Z'
Digit	= '0' NonZeroDigit
NonZeroDigit	= '1' '2' '3' '4' '5' '6' '7' '8' '9'
SpecialChar	= '+' '-' '*' '/' '%' '=' '!' '<' '>' '&' ' ' ':' ';' ',' '.' '(' ')' '[' ']' '{' '}' '"' '\'
OtherChar	= ? інший символ Unicode (лише в StringLit та Comment) ?

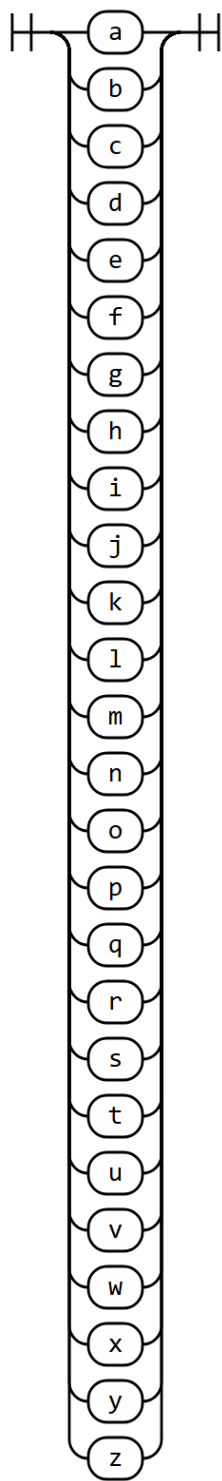
Alphabet



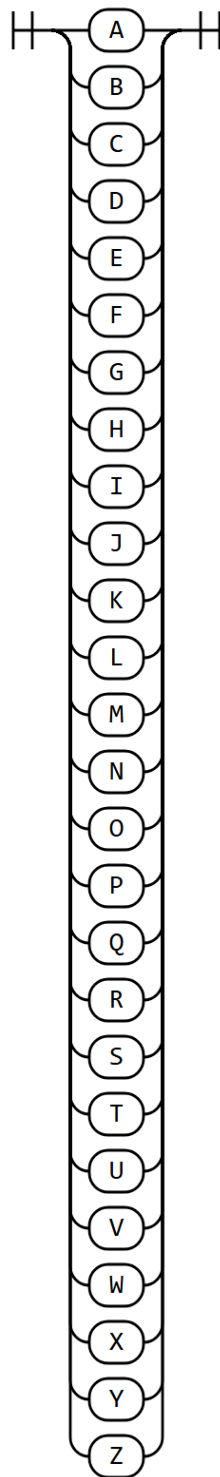
Letter



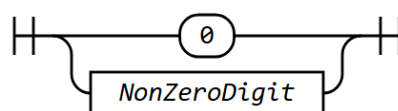
LowerLetter



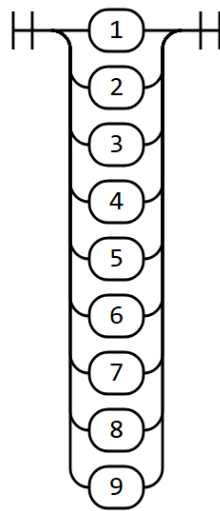
UpperLetter



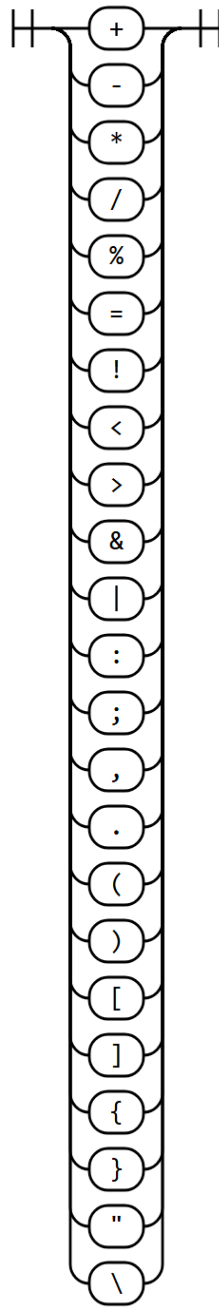
Digit



NonZeroDigit



SpecialChar



OtherChar

 ? *інший символ Unicode (лише в StringLit та Comment)* ? 

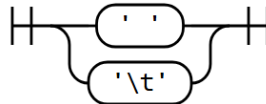
Рисунок 2.1 – Синтаксичні діаграми алфавіту

2.2 Роздільники рядків та пробільні символи

Пробільними символами є пробіл і горизонтальна табуляція. Роздільником рядків є символ нового рядка, якому може передувати символ повернення каретки (обидві форми, `'\n'` та `'\r' '\n'`, є допустимими).

```
WhiteSpace = ' ' | '\t'
EndOfLine  = '\n' | '\r' '\n'
```

WhiteSpace



EndOfLine

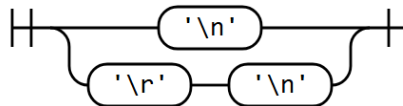


Рисунок 2.2 – Синтаксичні діаграми пробільних символів та роздільника рядків

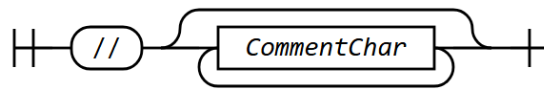
Пробільні символи та роздільники рядків не є токенами: вони лише розділяють лексеми і на цьому етапі вилучаються з потоку. Єдиний виняток — роздільник рядків, після якого за правилом розд. 2.9 може бути вставлено токен `;`. Кількість пробільних символів між токенами не обмежена і на зміст програми не впливає; відступи синтаксичного значення не мають.

2.3 Коментарі

Однорядковий коментар починається парою символів `//` поза рядковим літералом і продовжується до кінця рядка. Символ кінця рядка до коментаря не належить. Коментар вилучається лексичним аналізатором і діє як пробіл. Послідовність `//` усередині рядкового літерала коментарем не є. Багаторядкових коментарів мова не має.

```
Comment      = '//' { CommentChar }
CommentChar  = ? будь-який символ Unicode, крім '\n' та '\r' ?
```

Comment



CommentChar

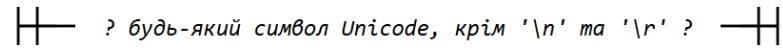


Рисунок 2.3 – Синтаксичні діаграми коментаря

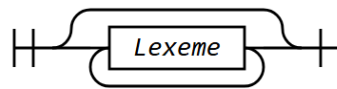
```
x := 10 // коментар до кінця рядка  
s := "// це не коментар"
```

2.4 Лексеми і токени

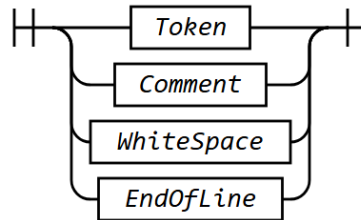
Текст програми є послідовністю лексем. Лексеми, що передаються синтаксичному аналізатору, називають токенами; це ідентифікатори, ключові слова, літерали, оператори та роздільники. Коментарі, пробільні символи та роздільники рядків токенами не є.

```
SourceText    = { Lexeme }  
Lexeme        = Token | Comment | WhiteSpace | EndOfLine  
Token         = Ident | Keyword | IntLit | FloatLit | StringLit | BoolLit  
              | Operator | Delimiter
```

SourceText



Lexeme



Token

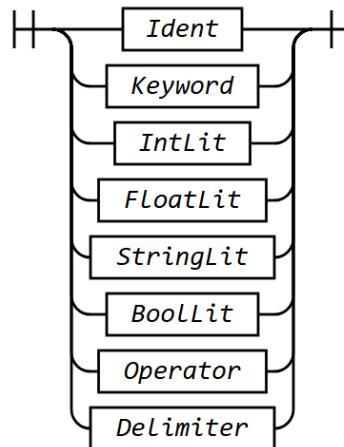


Рисунок 2.4 – Синтаксичні діаграми лексем

Між двома токенами, які інакше злилися б в один (наприклад, ідентифікатор і ключове слово), має бути принаймні один пробільний символ, роздільник рядків або коментар.

2.5 Ідентифікатори

Ідентифікатори позначають змінні, константи, типи, поля структур і функції. Ідентифікатор починається з літери або символу підкреслення, за якими можуть іти літери, підкреслення та цифри. Довжина ідентифікатора не обмежена, і всі його символи є значущими.

```
Ident      = Letter { Letter | Digit }
```

Ident

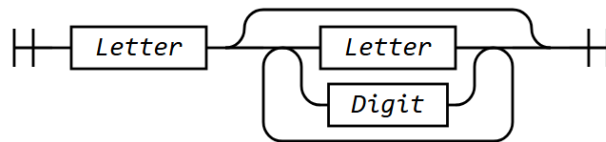


Рисунок 2.5 – Синтаксична діаграма ідентифікатора

Якщо лексема, що за формою могла б бути ідентифікатором, збігається з ключовим словом (розд. 2.6), вона є ключовим словом. Якщо вона збігається зі словом `true` або `false`, вона є логічним літералом.

Приклади: `x`, `total`, `Point`, `_tmp1`, `matrix2D`.

2.6 Ключові слова

Ключові слова зарезервовані й не можуть використовуватися як ідентифікатори.

Keyword	=	package		import		func		var		const		type		struct		
		if		else		for		return		int		float		bool		string
		len		fmt		'Print'		'Println'		'Scan'						

Keyword

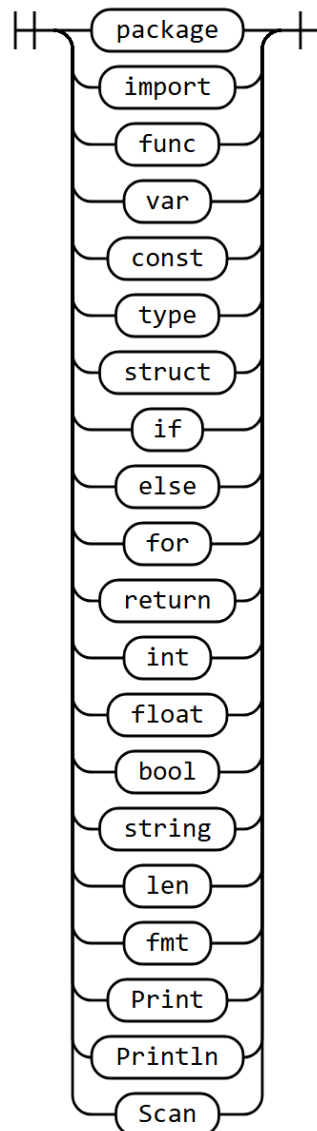


Рисунок 2.6 – Синтаксична діаграма ключових слів

Слова `int`, `float`, `bool` та `string` позначають базові типи, `len` — вбудовану функцію довжини. Слово `fmt` позначає вбудований пакет введення/виведення, а `Print`, `Println` і `Scan` — його операції. Слова `true` та `false` є логічними літералами (розд. 2.7).

2.7 Літерали

Літерали позначають сталі значення. Мова має літерали чотирьох видів: цілі, дійсні, рядкові та логічні; символічних літералів (окремого типу для одного символу) мова не передбачає. Тип літерала визначається його формою. Знак

числа до літерала не належить: у записі `-5` лексичний аналізатор виділяє два токени, оператор `-` і літерал `5`, а унарний мінус розпізнається під час синтаксичного аналізу.

2.7.1 Цілі літерали

Цілий літерал — десяткова послідовність цифр без знака. Щоб уникнути плутанини з вісімковими літералами мови Go, запис із ведучим нулем (наприклад, `007`) є лексичною помилкою; єдиний цілий літерал, що починається з `0`, — сам `0`. Цілий літерал має тип `int`, а його значення не може перевищувати `9223372036854775807`; інакше виникає помилка трансляції CE-10.

```
IntLit      = '0' | NonZeroDigit { Digit }
```

IntLit

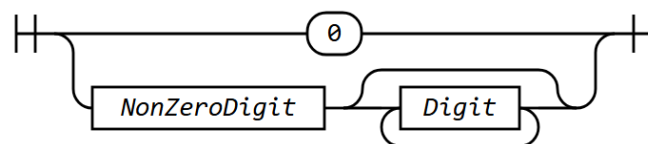


Рисунок 2.7 – Синтаксична діаграма цілого літерала

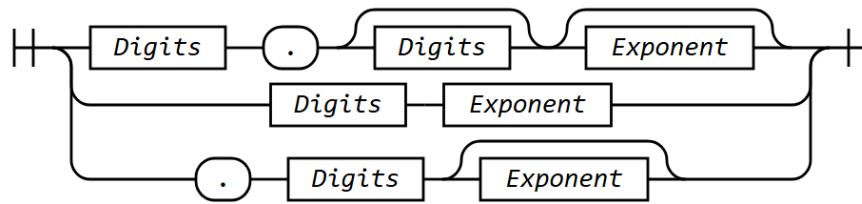
Приклади: `0`, `7`, `42`, `9223372036854775807`.

2.7.2 Дійсні літерали

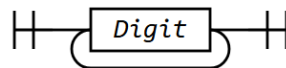
Дійсний літерал має дробову частину, експоненту або обидві. Цілу або дробову частину (але не обидві) можна пропустити. Дійсний літерал має тип `float`; значення, що після округлення до найближчого значення типу `float` дає нескінченність, є помилкою трансляції CE-10.

```
FloatLit    = Digits '.' [ Digits ] [ Exponent ]
              | Digits Exponent
              | '.' Digits [ Exponent ]
Digits      = Digit { Digit }
Exponent    = ( 'e' | 'E' ) [ '+' | '-' ] Digits
```

FloatLit



Digits



Exponent

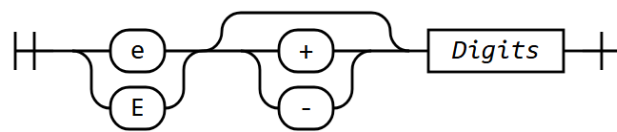


Рисунок 2.8 – Синтаксичні діаграми дійсного літерала

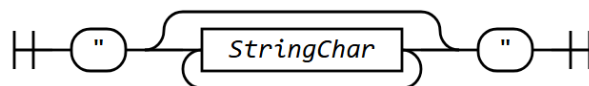
Приклади: 3.14, 2., .5, 1e3, 6.02E+23, 1.5e-3, 007.5.

2.7.3 Рядкові літерали

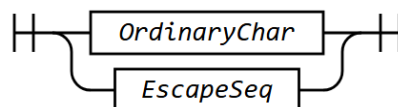
Рядковий літерал — послідовність символів у подвійних лапках. Він не може містити символ кінця рядка. Порожній рядок записується як `""`. Для запису спеціальних символів використовуються escape-послідовності (табл. 2.1); будь-яка інша послідовність, що починається зі зворотної косої риски, є лексичною помилкою CE-01.

```
StringLit    = '"' { StringChar } '"'
StringChar   = OrdinaryChar | EscapeSeq
EscapeSeq    = '\\' ( 'n' | 't' | 'r' | '"' | '\\' )
OrdinaryChar = ? будь-який символ Unicode, крім '"', '\\', '\n' та '\r' ?
```

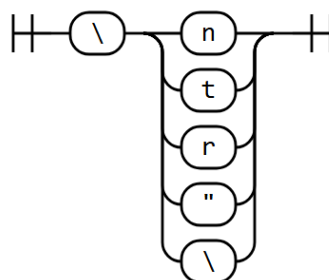
StringLit



StringChar



EscapeSeq



OrdinaryChar

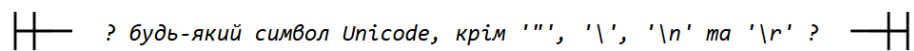


Рисунок 2.9 – Синтаксичні діаграми рядкового літерала

Таблиця 2.1 – Escape-послідовності рядкових літералів

Послідовність	Символ	Код Unicode
\n	новий рядок	U+000A
\t	горизонтальна табуляція	U+0009
\r	повернення каретки	U+000D
\"	подвійні лапки	U+0022
\\	зворотна коса риска	U+005C

Приклади: "", "Krok", "Привіт, світе!", "стовпчик 1\tстовпчик 2\n",
"лапки: \"x\"", коса риска: \\\

2.7.4 Логічні літерали

Логічні літерали true та false мають тип bool.

BoolLit = true | false

BoolLit

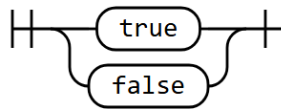


Рисунок 2.10 – Синтаксична діаграма логічного літерала

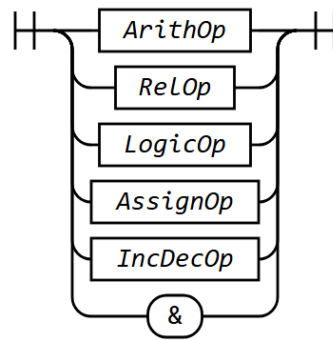
2.8 Оператори та роздільники

Оператори та роздільники — лексеми, утворені спеціальними знаками.

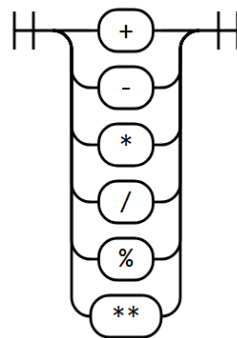
Нижче наведено їх повний перелік; семантику операторів описано в розд. 4.4.

Operator	=	ArithOp		RelOp		LogicOp		AssignOp		IncDecOp		'&'
ArithOp	=	'+' '-' '*' '/' '%' '**'										
LogicOp	=	'&&' ' ' '!'										
RelOp	=	'==' '!=' '<' '<=' '>' '>='										
AddOp	=	'+' '-'										
MulOp	=	'*' '/' '%'										
UnaryOp	=	'-' '+' '!'										
AssignOp	=	'=' ':='										
IncDecOp	=	'++' '--'										
Delimiter	=	'(' ')' '[' ']' '{' '}' ',' ';' '.'										

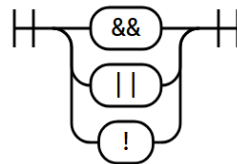
Operator



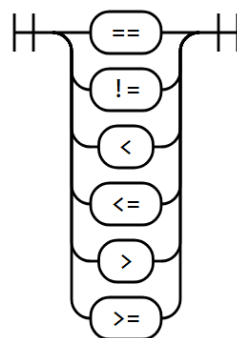
ArithOp



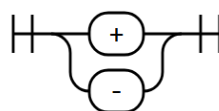
LogicOp



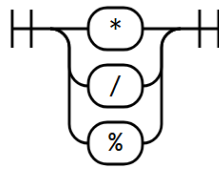
RelOp



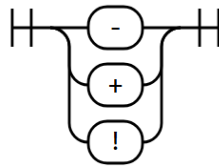
AddOp



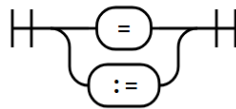
MulOp



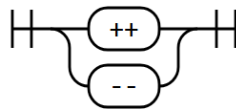
UnaryOp



AssignOp



IncDecOp



Delimiter

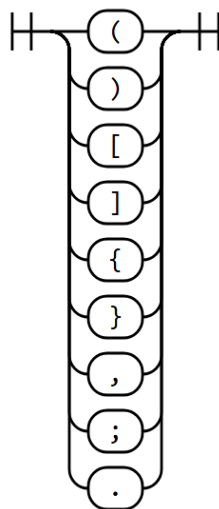


Рисунок 2.11 – Синтаксичні діаграми операторів та роздільників

Лексеми `+`, `-` та `*` мають кілька ролей, які розрізняє синтаксичний аналізатор за контекстом: `+` і `-` — бінарні або унарні оператори, `*` — множення, розіменування (`*p`) або позначка типу параметра-вказівника (`*int`). Правила

AddOp, MulOp та UnaryOp виділяють ті групи операторів, які використовуються у відповідних правилах граматики виразів.

2.9 Автоматична вставка крапки з комою

ГраMATика мови використовує токен `;` як завершувач оголошень та інструкцій. Як і в мові Go, у тексті програми крапку з комою зазвичай не пишуть: її вставляє лексичний аналізатор за таким правилом.

Коли лексичний аналізатор зустрічає роздільник рядків або кінець файлу, він вставляє в потік токенів `;`, якщо останній виділений у цьому рядку токен є одним із таких:

- ідентифікатор;
- цілий, дійсний, рядковий або логічний літерал;
- ключове слово `return`, `int`, `float`, `bool` або `string`;
- оператор `++` чи `--`;
- закриваюча дужка `)`, `]` або `}`.

Коментар до кінця рядка на правило не впливає. Явно записана крапка з комою дозволяє розмістити кілька інструкцій в одному рядку й обов'язкова в заголовку трикомпонентної інструкції `for`.

Наслідки правила:

- фігурна дужка, що відкриває блок, повинна стояти в тому самому рядку, що й `if`, `for`, `func` чи `struct`;
- ключове слово `else` повинно стояти в тому самому рядку, що й закриваюча дужка `}` попереднього блоку, інакше після `}` буде вставлено `;` і виникне синтаксична помилка;
- список елементів літерала масиву, розбитий на кілька рядків, має закінчуватися комою перед переходом на новий рядок.

```
if x > 0 {           // правильно
    y = 1
} else {             // правильно: else у рядку з }
    y = 2
}
for i := 0; i < 3; i++ { fmt.Print(i); fmt.Print(" ") } // явні ;
```

2.10 Таблиця токенів

Перелік усіх лексем мови Krok наведено в табл. 2.2. Таблицю побудовано за граматиною мови і вона містить усі термінали, що зустрічаються в її правилах; кожен запис відповідає рівно одній лексемі. Один приклад лексеми в рядку означає, що лексема унікальна; текст «символ ...» означає, що лексема є термінальним символом граматики. Саме ця таблиця є вхідними даними для побудови лексичного аналізатора.

Колонка «Клас» подає класифікацію лексем, прийняту в [2]: **TRM** — ключові слова, оператори та роздільники (термінальні лексеми, для яких достатньо коду); **IDN** — ідентифікатори; **CON** — константи (літерали). Лексеми класів IDN і CON, крім коду, потребують індексу у відповідній таблиці ідентифікаторів або констант, яку буде лексичний аналізатор. Останні чотири рядки таблиці (коментар, пробільні символи, роздільник рядків) позначено знаком «—»: вони не є токенами і синтаксичному аналізатору не передаються, але наведені для повноти опису лексичного рівня, оскільки роздільник рядків керує вставкою ; (розд. 2.9).

Таблиця 2.2 – Таблиця лексем мови Krok

Код	Приклади лексем	Токен	Клас	Неформальний опис
1	x, total, Point, tmp1	id	IDN	ідентифікатор
2	0, 42, 1000	intnum	CON	ціле без знака
3	3.14, 2., .5, 1e3, 6.02E+23	floatnum	CON	дійсне без знака
4	"" , "Krok", "a\tb\n"	strlit	CON	рядковий літерал
5	true	boolval	CON	логічне значення
6	false	boolval	CON	логічне значення
7	package	keyword	TRM	символ package
8	import	keyword	TRM	символ import
9	func	keyword	TRM	символ func
10	var	keyword	TRM	символ var
11	const	keyword	TRM	символ const
12	type	keyword	TRM	символ type
13	struct	keyword	TRM	символ struct
14	if	keyword	TRM	символ if
15	else	keyword	TRM	символ else
16	for	keyword	TRM	символ for
17	return	keyword	TRM	символ return
18	int	keyword	TRM	символ int
19	float	keyword	TRM	символ float

Код	Приклади лексем	Токен	Клас	Неформальний опис
20	bool	keyword	TRM	символ bool
21	string	keyword	TRM	символ string
22	len	keyword	TRM	символ len
23	fmt	keyword	TRM	символ fmt
24	Print	keyword	TRM	символ Print
25	Println	keyword	TRM	символ Println
26	Scan	keyword	TRM	символ Scan
27	+	add_op	TRM	додавання, унарний плюс, конкатенація
28	-	add_op	TRM	віднімання, унарний мінус
29	*	mult_op	TRM	множення; розіменування
30	/	mult_op	TRM	ділення
31	%	mult_op	TRM	остача від ділення
32	**	pow_op	TRM	піднесення до степеня
33	==	rel_op	TRM	дорівнює
34	!=	rel_op	TRM	не дорівнює
35	<	rel_op	TRM	менше
36	<=	rel_op	TRM	менше або дорівнює
37	>	rel_op	TRM	більше
38	>=	rel_op	TRM	більше або дорівнює
39	&&	and_op	TRM	логічне «і»
40		or_op	TRM	логічне «або»
41	!	not_op	TRM	логічне заперечення
42	=	assign_op	TRM	присвоєння
43	:=	define_op	TRM	оголошення з ініціалізацією
44	++	incdec_op	TRM	інкремент
45	--	incdec_op	TRM	декремент
46	&	addr_op	TRM	взяття адреси
47	(	paren	TRM	ліва кругла дужка
48	)	paren	TRM	права кругла дужка
49	[	bracket	TRM	ліва квадратна дужка
50	]	bracket	TRM	права квадратна дужка
51	{	brace	TRM	ліва фігурна дужка
52	}	brace	TRM	права фігурна дужка
53	,	punct	TRM	кома
54	;	punct	TRM	крапка з комою (явна або вставлена)
55	.	punct	TRM	крапка (доступ до поля, fmt.)
56	// коментар	comment	—	однорядковий коментар (ігнорується)
57	\32	ws	—	пробіл (ігнорується)
58	\t	ws	—	горизонтальна табуляція (ігнорується)
59	\n, \r\n	eol	—	кінець рядка (ігнорується; може породити ;)